

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 5: INHERITANCE, GENERALIZATION, SUPERCLASS, DAN SUBCLASS

Nama Project

: Sistem Informasi Akademik Mahasiswa (SIKAD)

Praktikan

: Mohamad Zaky Bahtiar Arifianto (3125522015)

Program Studi

: D3 Teknik Informatika - PENS Sumenep

Dosen Pengampu

: Nirwana Haidar Hari, S.Pd., M.Kom.

1. Sprint Goal — Modul 5

Sprint Goal: "Memperbaiki desain proyek dengan mengidentifikasi class yang memiliki karakteristik serupa dan melakukan generalization menggunakan inheritance." Melakukan audit menyeluruh terhadap arsitektur class P4, mengekstrak redundansi atribut dan method pada class Mahasiswa dan Dosen ke dalam sebuah superclass CivitasAkademika, menerapkan pewarisan menggunakan keyword extends dan constructor chaining via super(...), serta mempertahankan relasi objek (Association, Aggregation, Composition) yang telah dibangun.

2. Sprint Backlog (SB-01 s/d SB-07)

ID	Item Sprint Backlog	Deskripsi Luaran	Status
SB-01	Mencari class yang memiliki duplikasi	Mengaudit atribut identitas umum (nama, id, email) pada Mahasiswa dan Dosen	DONE
SB-02	Menentukan superclass	Mendefinisikan superclass CivitasAkademika sebagai induk entitas warga kampus	DONE
SB-03	Menentukan subclass	Menetapkan Mahasiswa dan Dosen sebagai turunan spesifik dari CivitasAkademika	DONE
SB-04	Memperbarui class diagram	Menyusun Class Diagram UML baru dengan notasi generalisasi (< --) dan relasi P4	DONE
SB-05	Implementasi extends	Menerapkan sintaks 'class Subclass extends Superclass' pada kode program Java	DONE
SB-06	Implementasi super	Menerapkan constructor chaining super(...) dan akses method superclass yang aman	DONE
SB-07	Melakukan pengujian inheritance	Menyusun Main.java untuk menguji instansiasi, method superclass & subclass	DONE

3. Audit Class & Pemetaan Generalisasi

Class Asal (P4)	Attribute Umum (Duplikasi)	Attribute Khusus (Spesifik)	Peran dalam Inheritance
Mahasiswa	• nomorInduk (NRP) • nama • email	• prodi, semester, ipk, totalSks • dosenWali (Association) • krs (Composition)	Subclass (Turunan dari CivitasAkademika)
Dosen	• nomorInduk (NIP) • nama • email	• bidangKeahlian	Subclass (Turunan dari CivitasAkademika)
CivitasAkademika (Hasil Refactoring)	• nomorInduk • nama • email • jenisCivitas	- (Menyediakan method umum: tampilkanIdentitas, accessor/mutator)	SUPERCLASS (Generalization Entity)

4. Class Diagram: Sebelum Refactoring vs Setelah Refactoring

A. Struktur Class P4 (Sebelum Refactoring - Tanpa Inheritance):

Pada P4, class Mahasiswa dan Dosen berdiri secara terpisah dengan atribut yang berulang (duplikasi nrp/nip, nama, email). Tidak terdapat hirarki pewarisan.

```
@startuml
class Dosen { - nip: String; - nama: String; - email: String; - bidangKeahlian: String }
class Mahasiswa { - nrp: String; - nama: String; - email: String; - prodi: String; - ipk: double; - dosenWali: Dosen; - krs: KRS }
Mahasiswa "1" *-- "1" KRS : composition >
Mahasiswa "0..*" --> "1" Dosen : association >
KRS "0..*" o-- "1..*" MataKuliah : aggregation >
@enduml
```

B. Struktur Class P5 (Setelah Refactoring - Dengan Inheritance):

Menerapkan Superclass CivitasAkademika. Class Mahasiswa dan Dosen meng-extends superclass (ditandai panah segitiga <|--). Seluruh relasi P4 (Association, Aggregation, Composition) tetap utuh dipertahankan.

```
@startuml
skinparam classAttributeIconSize 0
class CivitasAkademika {
- nomorInduk: String; nama: String; email: String; jenisCivitas: String
+ CivitasAkademika(nomorInduk: String, nama: String, email: String, jenis: String)
+ getNomorInduk(): String; getNama(): String; getEmail(): String; getJenisCivitas(): String
+ tampilkanIdentitas(): void
}
class Dosen extends CivitasAkademika {
- bidangKeahlian: String
+ Dosen(nip: String, nama: String, bidangKeahlian: String, email: String)
+ getNip(): String; getBidangKeahlian(): String; validasiDanSetujuiKrs(krs: KRS): void
}
class Mahasiswa extends CivitasAkademika {
- prodi: String; semester: int; ipk: double; totalSks: int
- dosenWali: Dosen
- krs: KRS
+ Mahasiswa(nrp: String, nama: String, email: String, prodi: String, sem: int, ipk: double, wali: Dosen)
+ getNrp(): String; hitungBebanMaksimalSks(): int; pilihMataKuliah(mk: MataKuliah): boolean
}
CivitasAkademika <|-- Dosen : extends (is-a)
CivitasAkademika <|-- Mahasiswa : extends (is-a)
Mahasiswa "1" *-- "1" KRS : composition >
Mahasiswa "0..*" --> "1" Dosen : association >
MataKuliah "0..*" --> "1" Dosen : association >
KRS "0..*" o-- "1..*" MataKuliah : aggregation >
@enduml
```

5. Penjelasan & Argumentasi Hubungan is-a

Penerapan inheritance pada proyek SIAKAD didasarkan pada pemenuhan prinsip **is-a** yang sah secara konseptual dan teknis:

1. Validasi Semantik Domain Pendidikan: Pernyataan bahwa "*Mahasiswa is-a CivitasAkademika*" dan "*Dosen is-a CivitasAkademika*" adalah kebenaran taksonomi akademik yang mutlak. Baik mahasiswa maupun dosen adalah bagian dari warga kampus (civitas akademik). Relasi ini **bukan** merupakan hubungan kepemilikan (*has-a*), karena seorang Dosen tidak memiliki Mahasiswa, melainkan keduanya berbagi karakteristik dasar kemanusiaan dan identitas institusi yang sama.

2. Pemenuhan Liskov Substitution Principle (LSP): Seluruh method dan sifat dasar yang didefinisikan pada superclass CivitasAkademika (seperti `getNomorInduk()`, `getNama()`, `getEmail()`, dan `tampilkanIdentitas()`) berlaku secara sah dan bermakna penuh bagi objek Mahasiswa maupun Dosen tanpa perlu mengubah semantik aslinya.

3. Pembedaan dengan Relasi has-a P4: Hubungan *has-a* tetap dipertahankan pada tempat yang tepat: Mahasiswa *has-a* KRS (Composition), KRS *has-a* MataKuliah (Aggregation), dan Mahasiswa *has-a* Dosen Wali (Association). Generalisasi ini memperjelas pemisahan antara **struktur hierarki jenis objek (is-a)** dengan **jaringan interaksi antar-objek (has-a)**.

6. Hasil Eksekusi Program (Screenshot & Log Running)

[TEMPAT SCREENSHOT HASIL RUNNING PROGRAM]

Eksekusi Main.java: `javac -d bin src/*.java && java -cp bin Main`

Verifikasi: Skenario 1 (Constructor Chaining via super), Skenario 2 (Method Superclass), Skenario 3 (Method Subclass), dan Skenario 4 (Integritas Relasi P4)

7. Bukti Penggunaan Keyword extends dan super()

- Keyword extends:** Diterapkan pada deklarasi class turunan: public class Mahasiswa extends CivitasAkademika dan public class Dosen extends CivitasAkademika. Keyword ini mewariskan seluruh atribut private dan method publik dari superclass secara langsung tanpa duplikasi kode.
- Keyword super() & Constructor Chaining:** Diterapkan pada baris pertama constructor subclass, misalnya pada Dosen: `super(nip, nama, email, "Dosen");` dan pada Mahasiswa: `super(nrp, nama, email, "Mahasiswa");`.
- Urutan Eksekusi Constructor:** Saat objek subclass diinstansiasi (contoh: `new Mahasiswa(...)`), Java secara otomatis mengeksekusi constructor superclass CivitasAkademika terlebih dahulu untuk menginisialisasi atribut fondasi (nomorInduk, nama, email), baru kemudian mengeksekusi tubuh constructor subclass Mahasiswa untuk atribut khusus (prodi, semester, ipk, pembuatan KRS).
- Akses Enkapsulasi:** Subclass tidak pernah mengakses atribut private superclass secara langsung, melainkan melalui method resmi superclass seperti `super.getNomorInduk()` dan `super.getNama()`.

8. Sprint Review — Modul 5

Item Evaluasi	Hasil Implementasi	Kendala & Solusi
Kandidat inheritance ditemukan	Ditemukan kesamaan karakteristik antara Mahasiswa dan Dosen (nomorInduk, nama, email).	Menganalisis nama atribut identitas yang berbeda (NRP vs NIP) agar dapat digeneralisasi.
Superclass berhasil dibuat	Class CivitasAkademika berhasil dibuat memuat 4 atribut umum dan method getter/setter/profil.	Tidak ada kendala, prinsip enkapsulasi dipertahankan murni.
Minimal 2 subclass dibuat	Class Mahasiswa dan Dosen berhasil difactor menjadi subclass dari CivitasAkademika.	Menyesuaikan constructor lama agar tetap kompatibel tanpa breaking changes.
extends diterapkan	Keyword extends berhasil diterapkan pada deklarasi class Mahasiswa dan Dosen.	Tidak ada kendala sintaksis.
super diterapkan	Constructor chaining via <code>super(...)</code> berhasil diimplementasikan pada seluruh subclass.	Menjamin pemanggilan <code>super(...)</code> diletakkan pada baris pertama constructor.
Class diagram diperbarui	Class diagram berhasil diperbarui dengan PlantUML menyajikan relasi inheritance & relasi P4.	Menggunakan simbol panah segitiga (< --) standar UML 2.5.
Program berhasil dijalankan	Program berhasil dikompilasi (javac) dan dieksekusi (java) dengan output 100% konsisten.	Bersih dari error dan warning.

9. Sprint Retrospective — Modul 5

What Went Well?	Refactoring generalisasi ke superclass CivitasAkademika berhasil mengeliminasi redundansi kode pada class Mahasiswa dan Dosen tanpa merusak struktur relasi objek P4 (Association, Aggregation, dan Composition). Constructor chaining menggunakan <code>super(...)</code> berjalan mulus dan pengujian method superclass/subclass di Main.java menghasilkan output yang terstruktur rapi.
What Went Wrong?	Tantangan awal muncul saat menyatukan penamaan nomor identitas yang berbeda antara NRP pada Mahasiswa dan NIP pada Dosen. Hal ini berhasil diatasi secara elegan dengan membuat atribut generik nomorInduk pada superclass, lalu menyediakan method delegasi <code>getNrp()</code> dan <code>getNip()</code> pada masing-masing subclass untuk menjaga kompatibilitas pemanggilan sebelumnya.
Improvement	Untuk sprint berikutnya (Modul 6: Polymorphism & Dynamic Method Dispatch), superclass CivitasAkademika dapat dideklarasikan sebagai abstract class dengan abstract method seperti <code>hitungHakAkses()</code> atau <code>cetakKartulIdentitas()</code> untuk memaksa setiap subclass menyediakan implementasi spesifik, sekaligus memanfaatkan dynamic binding secara maksimal.